IACR Transactions on Cryptographic Hardware and Embedded Systems
ISSN 2569-2925, Vol. 2025, No. 3, pp. 389–412. DOI:10.46586/tches.v2025.i3.389-412

Chameleon: A Dataset for Segmenting and Attacking Obfuscated Power Traces in Side-Channel Analysis

Davide Galli, Giuseppe Chiari and Davide Zoni

DEIB, Politecnico di Milano, Milan, Italy, firstname.lastname@polimi.it

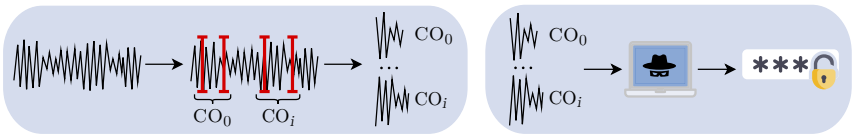
Abstract. Side-channel attacks exploit unintended information leakage emitted by cryptographic devices to extract sensitive data. Hiding techniques are a cost-effective countermeasure designed to obfuscate the side-channel leakage and hinder these attacks. Available open datasets rely on artificial models to simulate hiding effects, preventing a realistic assessment of these countermeasures and, thus, leaving a pressing need for datasets offering real-world, obfuscated side-channel measurements. Chameleon introduces the first comprehensive dataset of real-world, obfuscated power traces collected from a RISC-V-based System-on-Chip. The traces are obfuscated using four state-of-the-art hiding techniques: dynamic frequency scaling, random delay, morphing, and chaffing. Chameleon captures real leakage deformations introduced by actual hardware implementations, making it a realistic and valuable tool for evaluating side-channel countermeasures. A key feature of Chameleon is its dual focus on the segmentation and attack stages of the side-channel analysis process. It is the first dataset designed to facilitate the challenging task of segmenting cryptographic operations from obfuscated traces, offering precise metadata that pinpoints the start and end of each operation. The high-quality metadata enables systematic research into segmentation techniques, a critical step often overlooked in previous datasets. Chameleon provides an essential platform for researchers to develop and test new side-channel attacks, highlighting the vulnerabilities of current hiding techniques. By offering a more realistic assessment of countermeasure effectiveness, Chameleon is an invaluable tool for advancing the state-of-the-art in the side-channel evaluation.

Keywords: Side-Channel Analysis · AES · Hiding Techniques · Dataset · Deep Learning · RISC-V

1 Introduction

The ever-increasing number of interconnected computing devices deployed in every aspect of human activities, commonly known as the Internet of Things (IoT), calls for particular attention to their security requirements. Such requirements are traditionally fulfilled using cryptographic primitives implemented as either software libraries or dedicated hardware accelerators. However, one of the crucial aspects of providing security guarantees in the IoT scenario is the possibility for an attacker to seize and tamper with the device due to its embedded nature and field deployment. Consequently, side-channel attacks (SCAs) have become one of the most prominent security threats to modern IoT devices, as they are applicable even in a correctly implemented set of cryptographic primitives. Indeed, SCAs can exfiltrate sensitive information, e.g., the cryptographic secret key, from the computing device by correlating the partially known data being computed and a measured environmental signal produced by the device during the computation, e.g., the power consumption [KJJ99] or electromagnetic emissions [HMH+12].

Licensed under Creative Commons License CC-BY 4.0.
Received: 2025-01-15 Accepted: 2025-03-15 Published: 2025-06-05



(a) Segmentation stage. **(b)** Attack stage.

Figure 1: Side-channel analysis pipeline, divided into *segmentation* and *attack* stage.

Starting from the seminal work in [KJJ99], the importance of protecting IoT devices against SCAs motivated a vast literature targeting attacks [CRR03, BCO04, GBTP08] and defenses [CK09, ABPS15a, GMK16, DHL+21, KSM21] across the last two decades, and the use of machine learning is the current edge of the investigation [MPP16, BPS+20, RWPP21, GLMZ25]. Considering the wide variety of SCAs, all proposed techniques rely on two essential requirements: *i)* multiple executions of the same cryptographic operation (CO) with different inputs and *ii)* the precise identification and alignment of COs within the side-channel traces. SCAs typically follow a two-stage pipeline to meet these conditions, as depicted in Figure 1. First, the segmentation stage aims to identify and align the COs within the raw, noisy traces. Proper segmentation ensures that the relevant parts of the trace, i.e., the parts containing the information related to the execution of the COs, are correctly isolated and aligned. In a controlled laboratory environment, this task is facilitated by the use of security evaluation boards [Inr, Inc18], which provide trigger pins that allow the attacker to precisely synchronize cryptographic executions with their corresponding side-channel signals, greatly simplifying the segmentation process. However, segmentation becomes much more challenging in real-world scenarios where attackers deal with raw traces from real devices. Without access to trigger pins or other tools, the attacker must manually locate the COs within the side-channel trace, a task complicated by noise and signal variability. Second, the attack stage can begin once the COs have been properly segmented. Here, the attacker exploits the side-channel leakage embedded in the aligned COs to recover the secret key.

Modern countermeasures aim to harden both pipeline stages by desynchronizing the collected side-channel trace to enforce additional side-channel resistance [CK09, ABPS15a, DHL+21]. Desynchronization of the side-channel trace is achieved by so-called hiding techniques, which work by obfuscating the original side-channel signal by superimposing a random noise. By obfuscating the COs, hiding countermeasures prevent identification and isolation, i.e., segmentation, of the relevant portions of the trace and prevent precise alignment of the leakage points that are critical to the attack phase, thereby reducing the attacker's ability to extract meaningful information.

The literature contains several contributions targeting attack aspects of the obfuscation problem, while *i)* the large variety of available obfuscation countermeasures, *ii)* the time-consuming task of setting up a realistic experimental setup, and *iii)* the large amount of detailed data to be collected to set up a side-channel attack, prevented the use of a systematic approach, also preventing the comparison between different methodologies. It is crucial to have a realistic dataset that accurately reflects the temporal distortions introduced by these countermeasures to evaluate and validate the security of hiding techniques. The proposed attacks' practical effectiveness and reproducibility are significantly compromised without such a dataset. Currently, several open-source datasets [BPS+20, ANS21, rg14] offer aligned data collected from real-world microcontrollers, where obfuscation is artificially introduced through data augmentation. Data augmentation techniques such as *shift deformation* [CDP17, BPS+20], *add-remove deformation* [CDP17], and *random noise* [WP20] are commonly used to create obfuscated traces. While data augmentation can mitigate the lack of real-world data, verifying that the mathematical models used in these augmentations accurately represent the physical obfuscation phenomena is essential [PPM+23].

This manuscript addresses the critical need for a comprehensive dataset in side-channel research by introducing Chameleon, the first dataset designed to tackle the obfuscation introduced by four hiding countermeasures in side-channel traces. It provides the necessary foundation for systematic evaluation of both attack and segmentation techniques, offering raw power traces that capture the actual noise generated by real obfuscation mechanisms.

1.1 Related Works

The landscape of publicly available side-channel datasets is varied, offering different levels of complexity and protection mechanisms. Many datasets have become standards for evaluating attack methodologies, yet several limitations persist, particularly in modern, real-world implementations [PPM+23].

[rg14] and [CBT20] have been presented as part of the *DPAv4* and *CHES CFT* security contests, respectively. The DPAv4 dataset [rg14] contains 100 000 side-channel measurements of a masked version of AES-128 and AES-256 executed on an Atmel ATMega-163 smartcard. The CHES CTF dataset [CBT20] contains side-channel measurements collected from 50 000 executions of a masked AES-128 executed on the STM32 Cortex-M4 microcontroller. The ASCADv1 dataset [BPS+20] provides electromagnetic emission (EM) measurements from an ATmega8515 microcontroller executing a masked AES-128 implementation. The database offers two versions of the trace: one with a fixed key for 60 000 traces and another with 300 000 traces with random keys. Notably, the entire dataset has been collected from the ATmega8515 running at a fixed operating frequency, while the authors provide scripts to introduce artificial desynchronization. ASCADv2 [ANS21] introduces an additional 800 000 traces featuring random AES-128 encryptions protected with shuffling and affine masking. This dataset has been collected by measuring the power consumption of an STM32 Cortex M4 microcontroller running at a fixed operating frequency. Similarly, the AES_HD dataset [BJP20] contains 50 000 traces measuring the EM emissions of a hardware accelerator implementing AES-128. The hardware accelerator has been deployed on the SASEBO-GII evaluation board featuring a Xilinx Virtex-5 FPGA. The SMAesH dataset [CM24] offers power traces from a first-order masked hardware AES implementation on two FPGA platforms of different generations: a Xilinx Artix-7 FPGA and a Xilinx Spartan-6 FPGA. The dataset contains 6×2^{24} traces, capturing the execution of the first AES round. The eShard dataset [VTM21] provides EM leakage measurements from a software AES-128 implementation on an STM32F446 microcontroller. The implementation incorporates Boolean masking and shuffling to mitigate side-channel attacks. None of the publicly available datasets offer side-channel traces collected from a device implementing a hiding countermeasure to enforce temporal desynchronization. The sole exception, to the best of our knowledge, are AES_RD [Kiz20] and DFS_DESYNCH [GLMZ25] datasets. AES_RD represents a unique dataset containing 50 000 traces measured from an 8-bit Atmel AVR microcontroller implementing a random delay countermeasure, as discussed in [CK09]. Similarly, DFS_DESYNCH features a desynchronization countermeasure, comprising 256 000 power traces collected from a RISC-V CPU protected by a commercial DFS implementation with six randomly selected operating frequencies. However, the limitations of the available datasets are threefold, as highlighted in [PPM+23]. First, the datasets are limited in the number of supported hiding countermeasures, as only the AES_RD and DFS_DESYNCH datasets implement a single hiding technique. Second, all datasets are designed to tackle only the attack stage of side-channel analysis. Each trace in the datasets contains a single AES execution, eliminating the crucial step of locating each encryption from the raw measurements and with no possibility of investigating the pinpoint of the CO in the trace. Third, the side-channel traces exclusively contain COs, creating artificial scenarios in which a single application is executed in isolation and thus failing to capture the complexity of real-world systems, where cryptographic workloads coexist with general-purpose tasks.

1.2 Contributions

This paper proposes Chameleon, the first dataset to investigate the obfuscation problem in side-channel traces, tackling both the segmentation and attack stages of the side-channel pipeline. Chameleon consists of an extensive dataset of unprocessed real-world deformed power traces collected from a System-on-Chip that natively implements the four mainly employed hiding mechanisms, namely dynamic frequency scaling (DFS) [DHL+21], random delay (RD) [CK09], morphing (MRP) [ABPS15b], and chaffing (CHF) [ABPS15a]. Each trace contains several executions of the cryptographic operation of choice interleaved with multiple general-purpose applications, where the interleaving strategy has been selected to mimic the concurrent execution of the applications on the computing platform. The manuscript provides four contributions to the state-of-the-art:

- We introduce the first power traces dataset collected from a real target that natively implements four real-world hiding methods. In contrast to any previous dataset, the noise due to the actuators implementing the hiding countermeasure is measured rather than artificially modeled, making it a more realistic and valuable resource for assessing the effectiveness of side-channel countermeasures. Notably, the quality of the proposed dataset sits on the time-consuming acquisition process that requires a clean-room acquisition setup and system-on-chip. Without considering the design time to obtain the implementation of the computing platform and the working acquisition setup, the time required by the acquisition procedure exceeded 60 hours, delivering a dataset of 2 560 annotated power traces with a size of over 600 GB.
- We introduce the first dataset specifically designed to address both critical stages of the side-channel pipeline: segmentation, i.e., locating cryptographic operations, and the attack, i.e., exploiting leakage to recover the key. Previous datasets have focused solely on the attack stage, offering only pre-aligned traces. By providing metadata that accurately identifies the start and end of each cryptographic operation, Chameleon facilitates research of the challenging segmentation process, especially complex when dealing with obfuscated traces.
- This work is the first to provide side-channel segmentation methodologies under various obfuscation techniques. We present benchmarks to deal with traces obfuscated by real-world hiding countermeasures, revealing the significant work still needed to develop efficient segmentation methods in these complex scenarios.
- Finally, the quality of the Chameleon dataset has been validated by successful side-channel attacks. The attack results demonstrate that modern deep-learning attacks can bypass traditional hiding techniques, making it evident that relying solely on these techniques for security is no longer sufficient.

The Chameleon dataset is available online[1]. The rest of the paper is organized into five parts. Section 2 overviews the background, detailing the hiding countermeasures implemented, and the state-of-the-art techniques for benchmarking the segmentation and attack stages. Section 3 describes the infrastructure used to create Chameleon and the organization of data and metadata. Section 4 and Section 5 discuss the experimental validation of Chameleon for both segmentation and attack methodologies. Lastly, Section 6 discusses conclusions and future works.

2 Background

This section provides an overview of the hiding countermeasures implemented in Chameleon (see Section 2.1), the segmentation method used to locate the cryptographic operations in

[1] https://github.com/hardware-fab/chameleon

the power traces (see Section 2.2), and the deep-learning-based side-channel attack (see Section 2.3).

2.1 Hiding methods

The Chameleon dataset provides power traces obfuscated by four different hiding countermeasures, i.e., dynamic frequency scaling (DFS), random delay (RD), morphing (MRP), and chaffing (CHF). Notably, the decision to implement each hiding technique in hardware or software is guided by the traditional computer architecture design approach with the final goal of creating a dataset reproducing realistic scenarios.

Dynamic frequency scaling A randomized DFS continuously varies, in a random fashion, the clock signal frequency fed to the CPU, producing a distortion in the power consumption trace and reducing the information leakage [GGF+24]. The DFS mechanism has been hardware implemented in the SoC. We leverage a hardware true random number generator (TRNG) [GGFZ22] to provide a random number to the DFS actuator continuously. The random number selects a frequency from a pool of 760 available frequencies ranging from 5 MHz to 100 MHz, with a step of 125 kHz, thus enabling a large variability. Notably, DFS remained enabled throughout the entire acquisition phase. Consequently, a new frequency is requested as soon as a configuration is locked. As a result, each cryptographic primitive experiences a variable number of clock frequency reconfigurations, leading to significant trace deformation.

Random delay Random delay countermeasure introduces random delays in the execution of the cryptosystem to hinder the attacker's ability to correlate the power consumption with the executed instructions [CK09]. The random delay mechanism has been hardware implemented in the SoC. The CPU has been modified to implement the random delay mechanism at hardware level by leveraging a TRNG [GGFZ22] in the computing platform. At run-time, the TRNG keeps generating random numbers to determine the actual number (up to 2) of random instructions to be inserted between each pair of consecutive program instructions.

Morphing The morphing countermeasure leverages the TRNG [GGFZ22] to randomly modify the executed instructions of a cryptographic operation [ABPS15b]. The morphing mechanism has been software implemented. Morphing targets the AddRoundKey and SubBytes steps of the AES cryptosystem. Each AddRoundKey execution uses a randomly selected version of the XOR operation, chosen among the eight equivalent implementations with different power consumption profiles. SubBytes executions are instead morphed by employing two *S-Boxes* that are masked by random values and periodically refreshed.

Chaffing Chaffing countermeasure introduces a set of chaff keys, i.e., secret keys that are different but correlated with the actual secret key used to encrypt the plaintext, to confuse the attacker [ABPS15a]. Algorithm 1 describes the chaffing pseudocode. The chaff mechanism spawns multiple threads executing the same CO. Only one CO employs the correct secret key (see line 1 of Algorithm 1), while the others use the chaff keys (see line 3-5). The chaffing hiding mechanism is software implemented and leverages the TRNG [GGFZ22] embedded in the computing platform to support a randomized multi-threading scheduler at the operating system level. In particular, we implemented the randomized thread scheduler within the FreeRTOS real-time operating system [Fre24]. All the threads are started simultaneously (see line 6), until the thread performing the actual CO has completed (see lines 7-12), and their execution is interleaved according to a random scheduler. The Chameleon dataset contains side-channel traces using a chaffing

Algorithm 1 Chaffing countermeasure [ABPS15a].

```
Inputs: AES, key, ptx, numChaff
mainThread ← Create (AES, key, ptx)
chaffKey ← GenerateChaffKeys (key, numChaff)
for i in 1 to numChaff do
    chaffThreads[i] ← Create (AES, chaffKey[i], ptx)
end for
StartRandomScheduler (mainThread, chaffThreads)
while IsRunning (mainThread) do
    Wait ()
end while
for i in 1 to numChaff do
    Kill (chaffThreads[i])
end for
```

implementation employing three threads, i.e., using two chaff secret keys, to constrain acquisition time.

2.2 Segmentation methods

In a controlled lab environment, segmentation of CO is simplified using trigger signals, such as those provided by security evaluation boards (e.g., SASEBO SAKURA-II [Inr], NewAE CW305 [Inc18]) or virtual triggers via pattern-matching (e.g., Riscure icWaves [Ris20], NewAE ChipWhisperer Pro [New20]). However, real-world attacks lack such infrastructure, prompting methods like matched filters for AES detection [BFP22] and semi-automatic template-building techniques [TBW+21]. These works use the power consumption patterns of AES encryptions to identify COs in the traces, but they require a consistent template for all COs and manual intervention to build it. Despite successful AES detection, architectural countermeasures like multithreading, random delays, and DFS hinder CO localization, requiring more advanced detection techniques. Two seminal works rely on CNNs to segment COs in obfuscated traces by learning the power consumption patterns of the cryptographic operations [CGL+24, GCZ24]. The underlying intuition is to avoid a consistent manual template and let the CNN learn the best COs' template to handle the variability generated by hiding countermeasures. The generated template is then applied to a new side-channel trace to identify samples corresponding to a CO. [CGL+24] focuses on random delay hiding countermeasure, leveraging a CNN to distinguish COs from the remaining portions of the side-channel trace. The resulting segmentation is accurate enough to perform a successful side-channel attack. The authors extended their work in [GCZ24] to include dynamic frequency scaling techniques, adapting the deep-learning pipeline to the increased variability introduced by DFS.

2.3 Profiled deep-learning-based attack

Profiled deep-learning-based side-channel attacks aim to extract secret keys by analyzing power traces collected from cryptographic devices. These attacks operate assuming that the attacker has access to a clone of the target device, allowing them to gather power traces from known plaintexts and corresponding secret keys. Using this data, the attacker trains a deep-learning model to predict the secret key from new, unseen power traces. These attacks exploit correlations between power consumption and intermediate values of cryptographic algorithms, such as the *S-Box* in AES, to classify inputs into distinct intermediate value classes. The model minimizes prediction error during training, typically using categorical cross-entropy, ensuring reliable performance in side-channel domains [PPM+23].

Deep-learning SCAs commonly rely on multilayer perceptrons [PHJ+18, BPS+20] and CNNs [PHJ+18, BPS+20, ZBHV20], which excel in capturing patterns from side-channel data [PPM+23]. In the search for more efficient models, Rijsdijk et al. [RWPP21] applied reinforcement learning to optimize neural network hyperparameters for profiling SCAs. By rewarding the model based on attack performance, such as guessing entropy, the authors demonstrated an automatic exploration of the CNN architecture design space and the ability to generate models tailored for various cryptographic datasets.

Best practices for deep-learning-based SCAs include data standardization and normalization, which are essential steps in preparing input data for neural networks. Additionally, when profiling measurements are limited, data augmentation can improve model generalization. However, raw, unprocessed data is always recommended for training models in these attacks, as it provides the most direct representation of the power traces [PPM+23].

3 Chameleon dataset

Chameleon aims to complement the available open datasets by offering real-world, obfuscated power traces to the broader research community. While available datasets focus only on the attack phase, Chameleon has also been specifically designed to develop segmentation techniques. State-of-the-art datasets [BPS+20] usually employ mathematical models for desynchronization. However, Chameleon's side-channel traces leverage physical obfuscation processes, providing more realistic scenarios where the timing variations change unpredictably throughout the trace. Additionally, the obfuscation mechanisms generate genuine noise, particularly during transitions between different operating conditions, such as frequency shifts or delays, further amplifying this noise. The dataset is available at https://github.com/hardware-fab/chameleon.

The acquisition setup and the labeling methodology are discussed in Section 3.1. Section 3.2 details the dataset structure, and Section 3.3 provides statistics and Signal-to-Noise Ratio on the dataset.

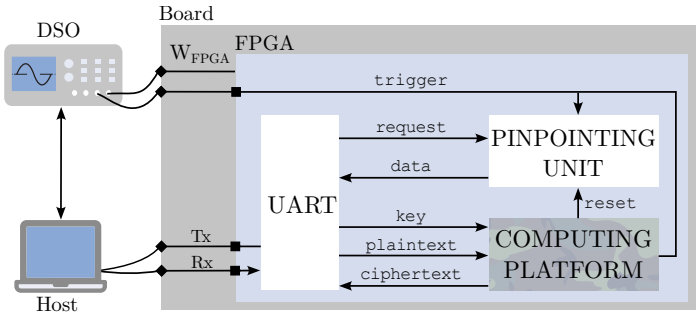


Figure 2: Block diagram of acquisition infrastructure.

3.1 Data acquisition

Figure 2 depicts the hardware-software infrastructure employed to acquire the dataset [ZGG25]. The setup comprises a host PC, a Picoscope 5244d digital sampling oscilloscope (DSO) [Tec21], and a NewAE CW305 [Inc18] board which hosts an AMD Artix-7 FPGA [Xil20]. A shunt resistor is located between the voltage supply and the FPGA to ease the measurement of the FPGA power consumption (W_{FPGA}). The FPGA implements a System-on-Chip (SoC) consisting of a 1.5 Mps UART interface to communicate with the host, a computing platform to execute the user applications, and a pinpointing unit to record the time sample of the beginning and the end for each CO in the traces. The computing platform implements

an in-order 32-bit RISC-V CPU [SZ19] that has been modified to implement the analyzed hiding methods. In particular, we provide a hardware implementation for the random delay and dynamic frequency scaling actuators, while morphing and chaffing techniques are software implemented. Within each measured side-channel trace, we perform multiple executions of the COs. For each execution of the CO, the computing platform takes as input the `plaintext` and the secret `key` from the host and sends back the computed `ciphertext`, which is checked for functional correctness by the host. To avoid polluting the measured side-channel signal with external noise, the plaintexts and the secret key are pre-loaded into the computing platform's memory before acquiring each side-channel trace. During this phase, the host PC also configures the DSO to ensure it is ready to capture the power traces corresponding to the upcoming cryptographic operations.

The synchronization between the computing platform, the DSO, and pinpointing unit is meticulously designed to ensure precise timing across these components, which is critical for accurate data collection. At the beginning of the acquisition of a new side-channel trace, a hardware `trigger` from the computing platform signals to start the acquisition of the power trace to the DSO. In parallel, the pinpointing unit is responsible for recording the instants when the computing platform begins and ends the execution of each cryptographic operation. The pinpointing unit takes as input the `reset` and `trigger` signal from the computing platform and a `request` signal from the host to send back the recorded `data`. To sinchronize the DSO acquisition with the pinpointing unit, the same trigger signal is fed to both the DSO and the pinpointing unit. The detailed description of the pinpointing unit is provided in Section 3.1.1.

As the cryptographic operation of choice, we selected the unprotected AES-128 cryptographic primitive from the OpenSSL [Tcl23] codebase. Notably, there are several other implementations of the AES algorithm. However, the wide adoption of the OpenSSL library in real applications allowed us to deliver a realistic dataset. The sampling rate of the DSO is set to 125 Msample/s with a resolution of 12 bits for the entire dataset. Notably, the CPU is clocked at 50 MHz for all acquisition campaigns, but for the DFS one for which the DFS actuator is instructed to randomly change the operating frequency of the computing platform as its maximum speed.

Algorithm 2 Pinpoint Recording

```
    Inputs: reset,
            trigger
1:  sample ← 0
2:  while trigger do
3:      sample ← sample + 1
4:      if reset commutates then
5:          Store(sample)
6:          sample ← 0
7:      end if
8:  end while
```

Algorithm 3 Frequency Recording

```
    Input: trigger
1:  sample ← 0
2:  while trigger do
3:      sample ← sample + 1
4:      if new frequency is stable then
5:          f ← GetFrequencyDFS()
6:          Store({sample, f})
7:          sample ← 0
8:      end if
9:  end while
```

3.1.1 Hardware labeling

To accurately label the beginning and the end of each AES execution, i.e., the COs, in each power trace, we developed an ad-hoc hardware module named pinpointing unit (see Figure 2). The module takes as input the `reset` and `trigger` signals from the computing platform and a `request` signal from the host. Following a host request, the pinpointing unit returns the recorded `data`.

Algorithm 2 explains the behavior of the pinpointing unit to record the measured encryption's first and last sample. When the computation is initiated, the CPU asserts

the `trigger` signal. The `trigger` signal starts the acquisition process on the DSO and simultaneously activates the pinpointing unit. As the computation progresses, the `sample` variable is incremented with each clock cycle until the CO execution starts, at which point the computing platform sends a `reset` signal to the pinpointing unit. This signal marks the beginning of the encryption process, storing the `sample` value. Once the encryption completes, the `reset` signal is de-asserted, marking the end of the process. In this way, the pinpointing unit records the first and last time encryption samples, ensuring that each label is accurately aligned with the corresponding cryptographic operation. The pinpointing unit operates based on a loop that iterates once per clock cycle, with a constant clock frequency. Similarly, the DSO's sampling frequency is maintained at a constant rate. This constancy ensures that the values recorded by the pinpointing unit are consistently correlated with the power trace samples captured by the DSO, with any differences being a simple multiplication factor. Once the computation is complete, the host PC retrieves all the data collected by the DSO and the pinpointing unit, ensuring that every trace is accurately aligned with the corresponding cryptographic operation.

As an additional metadata, the Chameleon dataset provides the frequency value for each sample of each side-channel trace. Using the DFS as a hiding technique, we designed a module that allows continuous and random changes in the CPU frequency during the computation. Then, we developed an additional hardware module to record the time of each frequency change and the actual frequency value. Algorithm 3 describes the behavior of the frequency recording unit. The `sample` variable is incremented at each clock cycle until the `trigger` signal stops the recording process. Notably, the clock cycle applied to the frequency monitoring module is kept fixed, i.e., it is not affected by the frequency changes. When the DFS actuator synthesizes a new frequency and becomes stable, the unit reads the current frequency from the DFS actuator and stores it along with the `sample` value. The `sample` variable is cleared. The complete hardware infrastructure has been added to the SoC, and a set of frequency resynchronizers is used to allow the coexistence of two operating frequencies, i.e., one dynamic for the CPU and the other fixed for the monitoring infrastructure.

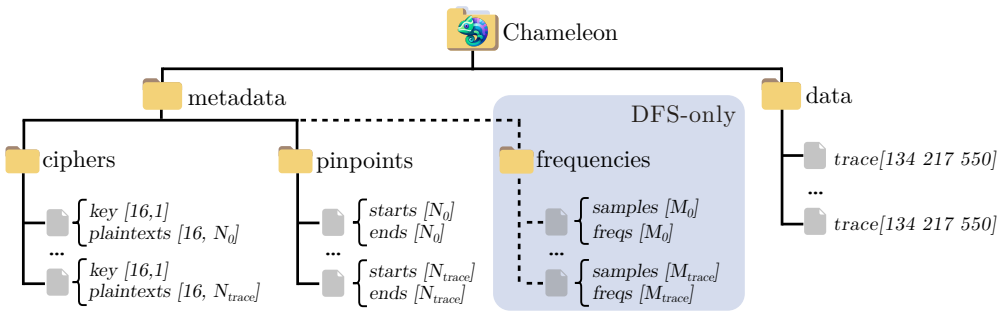


Figure 3: Dataset file system for each hiding method. `Frequencies` are available only for the `DFS` dataset.

3.2 Dataset structure

Chameleon dataset is organized into five subdatasets, i.e., `BASE`, `DFS`, `RD`, `MRP`, `CHF`, one for each hiding countermeasure implemented. `BASE` collects unprotected side-channel traces, while the other subdatasets are protected with a different hiding technique.

Figure 3 depicts the organization of the files in the dataset. The structure is the same for all the subdatasets, except for the `frequencies` folder, which is available only for the `DFS` part of the dataset. The dataset is organized in two folders: metadata and data.

Metadata For each hiding technique, the folder contains the information related to each execution of the COs within each trace. The metadata is divided into three sub-folders:

- `Ciphers` contains the plaintext and key of the AES executions in each trace file. The key is fixed for all the AES executions in the same trace, while the i-th plaintext is the output of the $(i-1)$-th encryptions to assure a random exploration of the plaintext space. The exact number of AES executions, thus plaintexts, in each trace i is different due to the random nature of the hiding mechanisms. We explored 256 different keys, namely all the possible variations of the first key byte keeping the last 15 bytes fixed, and the number of encryptions is balanced among the keys.

- `Pinpoints` contains the start and end time samples of each AES execution in each trace file. The value ranges from 0 to the trace length, i.e., 134 217 550 samples, and it is expressed in time samples. If an AES execution spans over the end of the trace, the end sample is set to the trace length, thus the last encryptions may be incomplete.

- `Frequencies`, only available for the `DFS` part of the dataset, contains the frequency changes due to the DFS actuations in each trace file. Each trace undergoes a different number of frequency changes due to the random nature of the DFS actuation, and the actual number of changes is given by M_i. The metadata compound two fields: *i) sample* marks the time sample when occurs a frequency change and ranges from 0 to the trace length; *ii) frequency* specifies the new operating frequency starting from the corresponding sample and takes a value from 5 to 100 MHz.

Data The folder contains N_{trace} raw measurements dumped directly from the oscilloscope. Each trace consists of 134 217 550 time samples equivalent to the storage capacity of Picoscope 5244d DSO [Tec21] operating at 125 MSample/s using a resolution of 12 bits. The side-channel trace is obtained from the SoC execution of the COs interleaved with general-purpose applications [CFG+20], i.e., an application different from a cryptographic one. The exact number of AES executions in each trace i is different due to the random nature of the hiding mechanisms, and it is given by parameter N_i in the metadata folder.

3.3 Statistics

Table 1 reports the statistics of the Chameleon dataset for each hiding method. The acquisition time is the total time required to collect the data for each hiding method. The `DFS` subset required the longest acquisition time of 43 hours and 20 minutes due to the labeling of the frequency changes. The number of data is the total number of traces collected and available in the dataset. Since we balanced the number of AES executions among the hiding methods, the number of traces, i.e., data, is different for each subset with a minimum of 256 traces for the `DFS` subset and a maximum of 1 024 traces for the `CHF` subset. The AES executions length is related to the implemented hiding mechanisms, with a mean ranging from 135 621 to 398 274 samples. The standard deviation is higher for the `CHF` subset due to the variable multithreading execution. The length of general-purpose applications is also reported, with mean and standard deviation. The executed general-purpose applications are randomly chosen from the same pool for every hiding mechanism except for chaffing. In `CHF` subset, the general-purpose applications consider the FreeRTOS routines of the scheduler and context switch.

Notably, each acquisition has been performed with a fixed CPU frequency of 50 MHz, except for the `DFS` subset. Therefore, the number of frequency changes is always one for each trace and the duration is the trace length, i.e., 134 217 550 time samples, except for the `DFS` subset. The DFS actuator performs a frequency change every 4 330 samples on

Table 1: Chameleon dataset statistics for each hiding countermeasure. Execution lengths and frequency durations are expressed in time samples.

		BASE	DFS	RD	MRP	CHF
Acquisition time		3h 31m	43h 20m	4h 17m	4h 15m	5h 28m
# Traces		256	256	512	512	1 024
# AES (per trace)	mean	821	898	536	575	262
	std	6.89	7.84	7.47	3.87	2.53
AES executions length	mean	148 192	135 621	220 120	195 599	398 274
	std	0.00	13 420.55	5 061.38	4 434.78	88 546.58
General purpose applications length	mean	15 276	13 695	30 096	37 789	112 878
	std	36 578.06	33 403.41	76 200.47	36 529.91	32 683.79
# Frequencies (per trace)	mean	1	31 000	1	1	1
	std	0	74.36	0	0	0
Frequencies duration	mean	134 217 550	4 330	134 217 550	134 217 550	134 217 550
	std	0	10.37	0	0	0

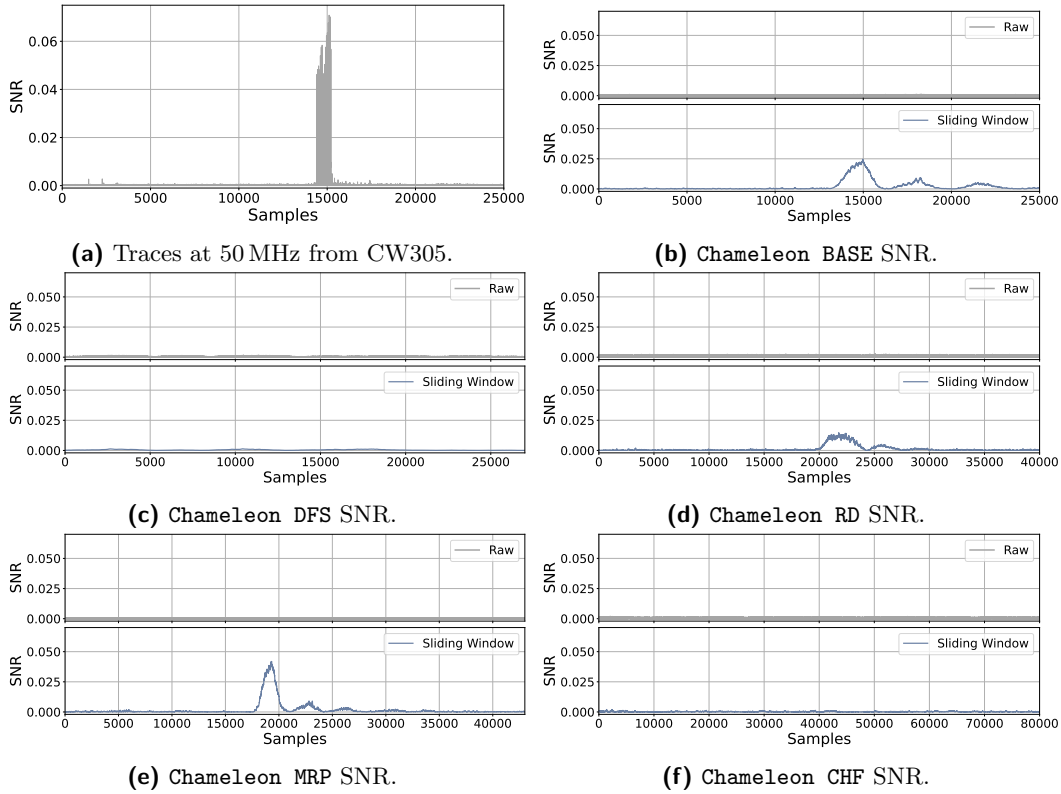
average, with a standard deviation of 10.37 samples. Thus, each DFS trace has a different number of frequency changes, with a mean of 31 000 changes.

3.3.1 Signal-to-Noise Ratio

A Signal-to-Noise Ratio (SNR) has been computed to quantify the noise level in the measurements. The analysis has been performed on raw traces and traces aligned using a sliding window of 100 samples [CCD00], aiming to mitigate the impact of hiding countermeasures. Since key byte 0 can vary across traces in Chameleon, the SNR is calculated for the intermediate `sbox(p[1]` $\oplus$ `k[1])` to identify the leakage samples associated with the *S-Box* computation of byte 1 during the first encryption round. Each SNR is computed with a total of 200 000 COs randomly sampled from Chameleon and correctly segmented using the metadata available in the dataset. As a reference, the SNR for unprotected AES traces, i.e., with a static frequency of 50 MHz and no countermeasures applied, and aligned leveraging the CW305's triggering infrastructure [Inc18, ZGG25], is reported in Figure 4a and referenced as CW305's SNR. Figure 4 plots each SNR in the first AES round interval. In each plot, the upper panel presents the SNR for raw traces, while the lower panel displays the SNR following sliding window alignment (SW-SNR). Since the length of each AES is random and different for each subdataset, we plotted the first fifth of the AES execution to ensure it always includes the first round's *S-Box* execution.

Comparing the CW305's SNR and the SNR on the raw `BASE` subdataset (see Figures 4a-4b), we can observe that the first-order leakages are significantly reduced in the Chameleon traces even without any countermeasure. The SNR reduction is likely due to Chameleon's long, single DSO acquisitions, capturing multiple consecutive COs. This prolonged capture increases vulnerability to environmental noise, which accumulates over time. In contrast, CW305's short, triggered acquisitions limit noise exposure, leading to a higher SNR. For the subdatasets that apply countermeasures (see upper panels in Figures 4c-4f for `DFS`, `RD`, `MRP`, and `CHF`), the SNR values on raw data are similarly low, without any peak. Each technique introduces variability and noise into the traces, effectively obfuscating the leakage and making the power traces harder to analyze.

After applying the sliding window alignment (see lower panels in Figure 4), the SW-



(a) Traces at 50 MHz from CW305.

(b) `Chameleon BASE` SNR.

(c) `Chameleon DFS` SNR.

(d) `Chameleon RD` SNR.

(e) `Chameleon MRP` SNR.

(f) `Chameleon CHF` SNR.

Figure 4: SNRs and SW-SNRs related to the intermediate `sbox(p[1]` $\oplus$ `k[1])` in the first AES round interval on raw traces and traces aligned with sliding window.

SNR values increase significantly for `BASE`, `RD`, and `MRP` subdatasets, reaching a peak value of 0.42 for `MRP`, thus mitigating environmental noise and lighter hiding countermeasures. However, there are no significant differences between raw SNR and SW-SNR for `DFS` and `CHF` subdatasets.

Notably, the SNR analysis aims to quantify the noise and leakage obfuscation in side-channel traces. For a thorough comparison of hiding techniques and dataset validation, please refer to Section 5, where the effectiveness of side-channel attacks confirms the Chameleon dataset's integrity.

4 Segmentation benchmarking

This section demonstrates the suitability of the Chameleon dataset for benchmarking segmentation techniques, dealing with the obfuscation of hiding countermeasures.

Figure 5 illustrates the two pipelines of the segmentation methodology. The Training Pipeline aims to develop a classifier for sorting a side-channel trace window into three categories: the start of a CO (class 0), the middle portion of a CO (class 1), or noise (class 2). The noise class is used to model the power traces containing no CO but general-purpose applications. The `Dataset Building` stage transforms the Chameleon's raw traces into a dataset of N-sample windows, leveraging the pinpoint metadata to compute and label each window. The obtained dataset is used to train a CNN classifier to discriminate between the three classes. The CNN architecture consists of six consecutive blocks: a convolutional block, two residual blocks, a global average pooling layer, a fully connected block, and a softmax layer. Each convolutional block includes a 1D convolutional layer, a

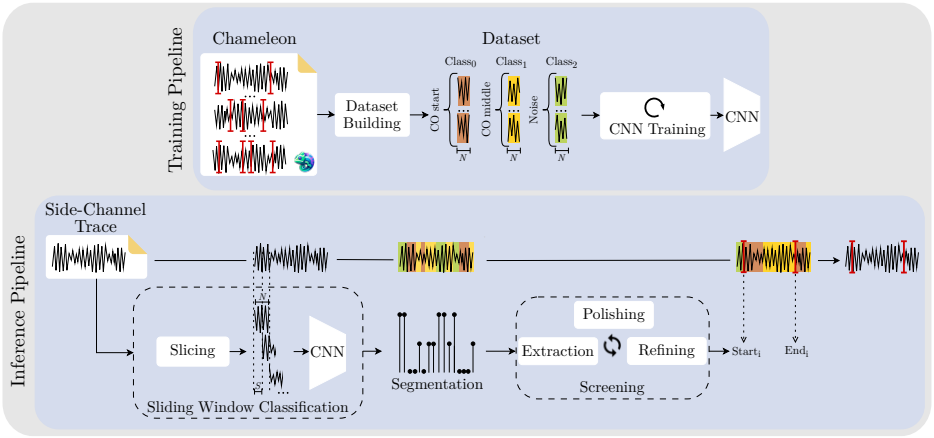


Figure 5: Methodology for segmenting cryptographic operation in Chameleon's obfuscated side-channel traces, divided into *training* and *inference* pipelines.

batch normalization layer, and a ReLU activation function. The residual blocks, which contain two convolutional blocks each, contain shortcut connections for element-wise feature summation. All convolutional layers employ a kernel size of 64, a stride of 1, and zero padding. The initial convolutional layer and the one in the first residual block use 16 filters, while the second residual block increases the filter count to 32. The global average pooling layer computes the average of the extracted features over the temporal dimension N. The resulting feature vector is then passed through two fully-connected layers, using a ReLU activation function, before being passed to the softmax layer, which generates a 3-dimensional vector containing the classification scores for each class.

Using the trained CNN, the Inference Pipeline is designed to detect COs within new side-channel traces. This pipeline has two main stages, namely, `Sliding Window Classification` and `Screening`. First, the `Sliding Window Classification` slices the side-channel trace into N-sample windows with a given stride value. Then, the CNN classifies each window as the start of a CO, a middle portion of a CO, or noise, producing a rough segmentation. Since the CNN's output might be noisy and not always suitable for directly determining the exact location of the COs, the `Screening` stage refines the segmentation and identifies the time instants that mark the boundaries of each CO. The `Screening` procedure used a majority voting mechanism to select the most likely class from a given kernel size and iteratively polishing the segmentation.

Heuristics for segmentation errors The Inference Pipeline may misidentify COs due to incorrect classification of a side-channel window or errors in extracting the start or end samples during the `Screening` procedure. These errors result in false positives, where a CO is identified but does not actually exist, or false negatives, where a CO is present but not detected. We introduce two heuristics to mitigate these issues, leveraging the dual identification of both the start and end sample of COs by the `Screening` process. For false positives, the heuristic pairs each CO start with its corresponding end, ensuring a one-to-one relationship and that each end always follows its start. If an unmatched start exists, it is flagged as erroneous, and the start that creates the shortest interval, unlikely to correspond to a valid CO execution, is removed. Similarly, unmatched ends are flagged and processed accordingly. For false negatives, the heuristic inspects intervals between pairs of CO start and end points. If an interval is significantly larger than expected for a single CO execution, it suggests that a CO operation may have been missed. In such cases, the interval is split, ensuring the new intervals are validated to confirm they do not

correspond to general-purpose executions.

4.1 Quality metrics

To assess the quality of the segmentation, we leverage two distinct metrics: intersection over union and false rate, i.e., *false positives* and *false negatives*.

Intersection over union Once all the COs in the data are localized, a standard metric to evaluate the quality of a segmentation is the *intersection over union* (IoU). It measures the overlap between the predicted and the ground truth segmentation. It ranges from 0 to 1, where 1 is a perfect overlap. It is formally defined by Equation 1 as the ratio of the intersection between the predicted (P) and the ground truth (GT) localization and their union.

$$IoU(P, GT) = \frac{|P \cap GT|}{|P \cup GT|} \tag{1}$$

False rates If CO executions are considered as a single object, *false positive* and *false negative rates* are two valuable metrics. A false positive is such if a CO is located even though it is not present in the given trace. A false negative is such if a CO is not located even though it is present in the given trace. False positive rate (FPR) and false negative rate (FNR) are defined as the proportion of false positives (false negative, respectively) relative to the actual number of COs.

4.2 Configurations

A distinct CNN model has been trained using a custom dataset for each Chameleon subdataset. The training of the CNN leverages an Intel Core i5-12400 with 64 GB of RAM and an NVIDIA GeForce GTX 1080Ti GPU, employing the PyTorch software programming framework. The training datasets were built by leveraging the pinpoint metadata, windowing the side-channel traces into N-sample windows, and labeling them according to three classes, namely start of a CO, middle of a CO, and noise. The window sizes were adjusted to approximate one round of the targeted CO, i.e., AES-128, as reported in Table 2. To mitigate bias during training, an effort was made to balance the three classes. However, there were insufficient N-sample windows for the noise class in some instances. In those cases, we opted to unbalance the classes rather than significantly reduce the dataset size. To address the imbalance, class weights were adjusted during loss minimization to compensate for the potential bias. We preprocess the power traces with a highpass filter with a cutoff frequency of 125 KHz to remove any low-frequency noise captured during the acquisition and with data standardization. Only 90% of the traces were used for training, while the remaining 10% were used for testing the segmentation. The training datasets were split again into training, validation, and testing sets, constituting 80%, 10%, and 10%

Table 2: Configuration parameters for the segmentation training and inference phase.

	BASE	**DFS**	**RD**	**MRP**	**CHF**
Window size	10k	10k	20k	20k	30k
Learning rate	0.007	0.007	0.007	0.007	0.007
Batch size	256	128	128	256	128
Drop out	0.30	0.45	0.4	0.30	0.35
Weight decay	$3e^{-5}$	$8e^{-5}$	$5e^{-5}$	$3e^{-5}$	$5e^{-5}$
Stride	50	50	100	100	100
Screening filter	150	400	300	150	160

Table 3: Results of side-channel segmentation on [CGL+24], [GCZ24], and this work on Chameleon.

		BASE	DFS	RD	MRP	CHF
[CGL+24]	**IoU** (%)	–	–	–	–	–
	FNR (%)	0.22	4.40	3.08	0.34	1.22
	FPR (%)	0.59	6.76	0.00	12.44	0.54
[GCZ24]	**IoU** (%)	–	–	–	–	–
	FNR (%)	0.01	0.06	1.15	0.00	0.67
	FPR (%)	0.11	0.73	0.13	0.00	0.48
This work	**IoU** (%)	89.47	88.23	84.19	91.26	87.96
	FNR (%)	0.00	0.00	0.00	0.00	0.00
	FPR (%)	0.00	0.00	0.00	0.00	0.00

of the total. Each CNN network underwent training for 75 epochs, leveraging the Adam optimizer paired with the one-cycle learning rate scheduler.

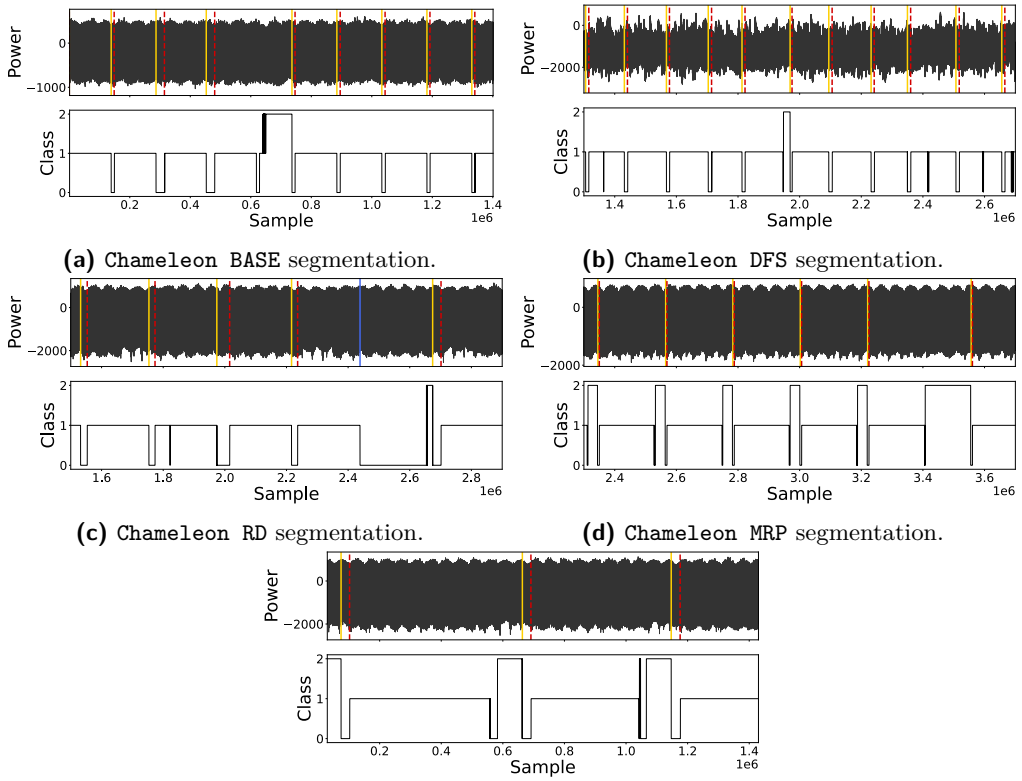
A comprehensive hyperparameter search has been performed, inlcuding the learning rate, the batch size, and the regularization values, using a random search strategy [BB12]. Table 2 reports the choosen CNN hyperparameters used for training and the input window size. The inference parameters, namely, the stride used in the `Sliding Window Classification` and the screening filter for polishing the segmentation, are also reported in Table 2.

4.3 Results

Last rows of Table 3 report the segmentation results for the Chameleon dataset. The results indicate that all segmentation techniques achieved a high IoU, ranging from 84.19% to 91.26%. The morphing technique consistently outperformed others, suggesting it introduces less noise and variability into the traces. The IoU could be further improved by decreasing the stride in the `Sliding Window Classification`, albeit at the cost of increased computation time. The heuristics introduced to mitigate segmentation errors have been effective, consistently reporting 0% FNR and FPR across all techniques and demonstrating excellent performance in detecting all the present COs.

Figure 6 helps to better understand the segmentation results by showing the CNN models' output on the Chameleon dataset. All the plots show random portions of the traces but with the same number of samples to allow a comparison of the density and length of the AES operations for each hiding technique. For instance, Figure 6c shows the `RD` segmentation struggling with the class 2, i.e., the noise class, often misclassifying it as class 0, i.e., the start of a CO. This misclassification can be noticeable by looking at the classifications starting with sample 2.41e6, which are classified as class 0 without having a ground truth reference. This classification error leads to the generation of a false positive, highlighted by the blue vertical line and correctly fixed by the heuristics. Notably, the segmentation towards the end of the COs is rougher, as clearly visible in Figure 6e. This behavior is expected, as the CNN model has been trained to detect the start of the COs, not their end. However, the end of the COs can be easily identified by looking at the segmentation plots, as the COs are always followed by a period of noise or the start of a new CO. By comparing the position of the ground truth and the position of the CO found, it can be seen that the latter is always a few hundred samples earlier. This conservative approach was taken to ensure the inclusion of all samples belonging to a CO at the expense of a lower IoU in the segmentation.

Table 4 provides the confusion matrices over the test set for each CNN model, high-



(a) `Chameleon BASE` segmentation.

(b) `Chameleon DFS` segmentation.

(c) `Chameleon RD` segmentation.

(d) `Chameleon MRP` segmentation.

(e) `Chameleon CHF` segmentation.

Figure 6: Segmentation results for the Chameleon dataset in random sample interval. In the top panels, red vertical lines represent the start of the COs' ground truth, while yellow vertical lines represent the predicted CO's starts. The bottom panels show the output classification.

lighting high classification accuracy on the main diagonal. Nevertheless, the excellent classification in the training phase is not always reflected in the inference phase. For instance, `RD` achieves over 99.99% accuracy in training but exhibits misclassification for classes 2 and 0 during inference in Figure 6c. The lower inference performance is probably due to the sliding window procedure adopted, which increases the input variability to the CNN, challenging the classification.

These results confirm the feasibility of developing effective segmentation techniques on the Chameleon dataset. The high IoU values achieved on various hiding techniques demonstrate that it is possible to accurately identify COs within obfuscated traces, even in the presence of significant noise and variability.

Comparison with related works This study builds upon and enhances seminal techniques [CGL+24, GCZ24] by accurately identifying both the start and end samples of CO executions, yielding two significant advantages. First, it enables the calculation of the IoU metric, a critical measure of segmentation accuracy. In contrast, prior works only detect the start of encryption, precluding a precise computation of IoU. Second, by delineating both start and end samples, Chameleon enhances the robustness of the proposed segmentation methodology, enabling a more accurate identification of false positives and false negatives and using two heuristics to mitigate such errors.

Table 3 presents a comparative analysis of our side-channel segmentation performance

Table 4: Test confusion matrices for the segmentation of the Chameleon dataset.

(a) `Chameleon BASE`

Predicted class \ True class	0	1	2
0	**100%**	0%	0%
1	0%	**100%**	0%
2	0%	0%	**100%**

(b) `Chameleon DFS`

Predicted class \ True class	0	1	2
0	**99.74%**	0.25%	0.01%
1	0.11%	**99.89%**	0%
2	0%	0%	**100%**

(c) `Chameleon RD`

Predicted class \ True class	0	1	2
0	**99.99%**	0.01%	0%
1	0.01%	**99.99%**	0%
2	0%	0%	**100%**

(d) `Chameleon MRP`

Predicted class \ True class	0	1	2
0	**100%**	0%	0%
1	0%	**100%**	0%
2	0%	0%	**100%**

(e) `Chameleon CHF`

Predicted class \ True class	0	1	2
0	**98.95%**	0%	1.05%
1	0%	**99.91%**	0.09%
2	1.00%	0.31%	**98.69%**

against two prior works [CGL+24, GCZ24], using the Chameleon dataset. FNR and FPR are reported for all techniques, while IoU values are exclusive to this study due to its ability to pinpoint both start and end samples, a distinction that precludes fair IoU comparisons with earlier works. Although [CGL+24] has been developed to tackle only random delay, it performs well across all hiding techniques, particularly on `BASE` and `CHF`. Similarly, [GCZ24] demonstrates promising results, achieving improved performance across all metrics. However, this study is the only one with perfect FNR and FPR scores for all hiding countermeasures. The excellent results are attributed to the novel tailored heuristics for error refinement, showcasing the robustness and precision of the proposed segmentation methodology.

Limitations The proposed framework uses a window-based classification to segment COs in side-channel traces. A window-based classification means that the CNN classifies each window independently, resulting in a sequence of classifications rather than a proper segmentation. While our methodology effectively detects COs with high IoU and low false rates, future work will focus on developing a more accurate segmentation technique to classify each individual sample within a side-channel trace as part of a CO or not.

By achieving a more granular segmentation, we can refine attack strategies and improve the overall effectiveness of real-world side-channel analysis.

5 Attack benchmarking

To assess the quality of the Chameleon dataset, we aligned and successfully attacked the collected COs, grouped by hiding method. The attack phase was evaluated separately from the segmentation phase to perform a comprehensive dataset validation and assess the correct acquisition of side-channel information. Specifically, we conduct a CPA attack [BCO04] with a sliding window aligment [CCD00] (SW-CPA) and a deep-learning-based attack employing the tool introduced in [RWPP21] to generate high-performance CNN architectures automatically. The [RWPP21] methodology uses reinforcement learning

to sequentially select CNN layers and hyperparameters guided by a reward function optimized for SCA performance. This reward function evaluates the network based on its guessing entropy performance, including how quickly the guessing entropy converges to 1 and considering network validation accuracy. The reward function prioritizes models that can extract the secret key using fewer traces, optimizing efficiency and attack success. The algorithm is customized to focus on neural architectures commonly effective in SCA, with the search space consisting of convolutional, average pooling, batch normalization, or global average pooling layers. Each CNN ends with a softmax layer preceded by flattened or fully connected layers using the SeLU activation function.

5.1 Quality metrics

A side-channel attack can be carried out after locating and isolating the various ciphers. To assess the attack effectiveness, we consider three metrics: the guessing entropy, the number of executions, and the CNN success rate.

Guessing entropy The guessing entropy (GE) is defined as the average rank position of the correct key among all possible key guesses, namely, the number of guesses required to find the correct key. The best possible GE value is 1, meaning the key is always precisely predicted.

Number of executions The metric *number of executions* expresses the number of CO required to reach a guessing entropy of 1. For SW-CPA, we consider the *number of executions* needed to reach a GE of 1 for all attacked bytes. For CNN-based attacks, we consider the *number of executions* of the best CNN model, i.e., the one that requires the fewest COs to recover the key.

CNN success rate The robustness of a countermeasure is inversely related to the number of CNNs that successfully recover the key. The more CNNs are able to achieve key recovery, the less resistant the countermeasure is considered to be. The metric *CNN success rate* refers to the percentage of tested CNNs that reached a guessing entropy of 1, highlighting how easily different neural network configurations can exploit the leakage. Notably, a *CNN success rate* greater than 0 is enough to recover the key.

5.2 Configurations

All the attacks exploit the leakage due to the *S-Box* substitution on the first AES round. Since we want to assess the dataset's quality, we leverage the Chameleon metadata to perform the COs' localization step. Each encryption has been truncated to include the first AES round and to fix the input size for the attack. Specifically, DFS, random delay, and morphing traces have been truncated after 100 000 samples, while chaffing traces after 200 000 to take into consideration chaffing higher length variability.

The SW-CPA attack uses a sliding window of 100 samples, and it is computed with a total of 200 000 COs randomly sampled from Chameleon. The attack is performed on the last 15 bytes of the AES key, as the first byte is not fixed.

In the CNN-based attacks, we preprocess the raw power traces with a highpass filter with a cutoff frequency of 125 KHz to remove any low-frequency noise captured during the acquisition. Moreover, every trace underwent sub-sampling by averaging 100 consecutive samples to reduce the input size and perform faster training without compromising the side-channel information [PWP22]. We target the first byte during the profiling and attack phase. As in standard deep-learning attacks, we split the dataset in training, validation, and test sets with a ratio of 80%, 10%, and 10%, respectively. The test set is used to

Table 5: Results of the SW-CPA and CNN-based attacks on the aligned AES executions.

		BASE	DFS	RD	MRP	CHF
SW-CPA	**GE**	1	121.76	1	1	118.20
	# Executions	$4.7k$	–	$37.5k$	$14k$	–
CNN-based	**GE**	1	1	1	1	1
	# Executions	14	79	100	28	104
	CNN success rate (%)	48	0.7	1.2	3.0	0.19

evaluate the performance of the attack on the aligned traces. Key values are balanced from the set of possible keys for training and testing, and the COs are shuffled to avoid bias in the model learning process. We train 500 models for each hiding method on an Intel Core i5-12400 with 64 GB of RAM and an NVIDIA GeForce GTX 1080Ti GPU. We set the number of epochs to 50, the batch size to 256, and a one-cycle learning rate scheduler applied to the Adam optimizer.

5.3 Results

Table 5 shows the attack's results on the aligned AES executions across the five different Chameleon subdatasets. For the SW-CPA attack, results are reported in terms of the guessing entropy (GE) and the number of COs required to recover the secret key. The results demonstrate that the `BASE` subdataset, which lacks any countermeasures, allows efficient key recovery with a GE of 1 achieved after only $4.7k$ COs. The `MRP` and `RD` subdatasets also allow key recovery, requiring $14k$ and $37.5k$ COs, respectively, indicating that these countermeasures increase resistance to SW-CPA but do not entirely prevent it. In contrast, for the `DFS` and `CHF` subdatasets, the attack does not succeed within the attacked dataset size, highlighting their effectiveness in obfuscating side-channel leakage.

For the CNN-based attack, the performance of the attack is evaluated by averaging 256 independent attacks, i.e., one for each key byte value. Along with GE and number of executions, we introduce a third metric, the percentage of effective CNN models trained, to a more robust comparison between countermeasures. The GE is 1 across all tested scenarios, but the number of executions needed to achieve it varies significantly between subdatasets. Notably, the unprotected `BASE` scenario requires only 14 executions to recover the key, demonstrating its vulnerability. When the chaffing countermeasure was applied, the number of executions rose sharply to 104, making it appear the most resilient. However, we stopped the CNN generator after training 500 models for each scenario, leaving potentially better-performing models unexplored. For this reason, the number of COs required to recover the key should not be solely used to compare the different hiding techniques. The *CNN success rate* reveals a clearer picture of the relative robustness of the countermeasures. As seen in the last row of Table 5, for the `BASE` scenario, 48% of the trained CNNs were able to recover the key, confirming that unprotected AES implementations require little effort in model tuning to exploit the leakage. The effectiveness drops dramatically for subdatasets with countermeasures. Specifically, `DFS` and `CHF` show success rates of only 0.7% and 0.19%, respectively, demonstrating their ability to reduce leakage significantly. The `RD` and `MRP` subdatasets exhibit higher success rates, at 1.2% and 3.0%, respectively, reflecting the SW-CPA results.

Overall, the results confirm the high quality of the Chameleon dataset. Successful key recovery demonstrates that the COs are correctly located in the traces as indicated by the metadata and that the computations align with the provided plaintexts and keys. Furthermore, the ability to perform both SW-CPA and CNN-based attacks validates the accuracy of the side-channel trace acquisition, ensuring the dataset can effectively support the development of new attack methodologies.

5.4 Segmentation errors impact

The attack phase was evaluated under the assumption of perfect segmentation to validate the dataset. However, identification errors can occur during segmentation, potentially affecting attack performance. Although the segmentation technique and error-handling heuristics detailed in Section 4 demonstrated excellent results on the Chameleon dataset, achieving a 0% FNR and FPR in real-world scenarios is impractical. Such identification errors can significantly impact attack effectiveness by causing mismatches between COs and their corresponding plaintexts. In a naive matching approach, the position of the error significantly affects its impact. If a mismatch occurs early in the sequence, it causes a cascading misalignment for all subsequent CO executions. On the other hand, mismatches that occur later in the sequence are less disruptive since all preceding traces remain correctly matched with the corresponding plaintexts. However, for deep learning-based profiled attacks, the impact of identification errors is minimal, as such attacks typically require very few traces to succeed. For instance, even with an identification error in the 15th CO, the correct key may already have been recovered using the first 14 CO executions. Classical unprofiled attacks are more susceptible, as they generally depend on a higher number of correctly matched COs.

To quantitatively assess the impact of identification errors on Chameleon's attack performance, we considered a 0.05% error rate. For the most challenging subdataset, i.e., `CHF`, which requires 104 traces for a successful CNN-based attack, the probability of attack failure is 5.07%. In comparison, for the `BASE` subdataset, requiring only 14 traces, the failure probability decreases to 0.70%.

6 Conclusions

This work introduces Chameleon, a novel dataset designed to evaluate the efficacy of obfuscation countermeasures against side-channel attacks. Chameleon offers a distinct advantage over existing datasets by providing real-world, unprocessed power traces captured from a System-on-Chip that natively implements four widely used hiding mechanisms, namely, dynamic frequency scaling, random delay, morphing, and chaffing. The side-channel power traces capture the authentic noise introduced by the hardware and software actuators, allowing for a more realistic assessment of the countermeasures' effectiveness compared to datasets that rely on artificially simulated noise. Additionally, Chameleon is the first dataset that addresses both the segmentation and attack stages of the side-channel analysis pipeline. Tackling the whole pipeline is a critical contribution, as prior datasets focused exclusively on the attack phase, ignoring the challenging task of segmenting cryptographic operations from noisy, obfuscated traces. Chameleon facilitates segmentation research by providing precise metadata that indicates the start and end of each cryptographic operation, thus enabling systematic exploration of segmentation techniques for side-channel traces. Our benchmarking results demonstrate the limitations of current hiding techniques, showing that deep-learning-based attacks are capable of bypassing them.

While this study provides insights into the resilience of state-of-the-art hiding techniques, the detailed analysis of these countermeasures in the context of masked ciphers is an important and complementary area of research. Exploring the effectiveness of these hiding techniques in secure cryptographic implementations is an open direction for future research.

Chameleon will be a valuable resource for the side-channel community, enabling researchers to develop and evaluate novel segmentation and attack techniques against real-world, obfuscated power traces.

References

[ABPS15a] Giovanni Agosta, Alessandro Barenghi, Gerardo Pelosi, and Michele Scandale. Information leakage chaff: Feeding red herrings to side channel attackers. In *2015 52nd ACM/EDAC/IEEE Design Automation Conference (DAC)*, pages 1–6, 2015.

[ABPS15b] Giovanni Agosta, Alessandro Barenghi, Gerardo Pelosi, and Michele Scandale. The meet approach: Securing cryptographic embedded software against side channel attacks. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 34(8):1320–1333, 2015.

[ANS21] ANSSI (Agence Nationale de la Sécurité des Systèmes d'Information). Ascadv2, 2021. https://www.data.gouv.fr/en/datasets/ascadv2/.

[BB12] James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *J. Mach. Learn. Res.*, 13(null):281–305, feb 2012.

[BCO04] Eric Brier, Christophe Clavier, and Francis Olivier. Correlation power analysis with a leakage model. In *Cryptographic Hardware and Embedded Systems-CHES 2004: 6th International Workshop Cambridge, MA, USA, August 11-13, 2004. Proceedings 6*, pages 16–29. Springer, 2004.

[BFP22] Alessandro Barenghi, Gioele Falcetti, and Gerardo Pelosi. Locating side channel leakage in time through matched filters. *Cryptography*, 6(2), 2022.

[BJP20] Shivam Bhasin, Dirmanto Jap, and Stjepan Picek. AES HD dataset - 500 000 traces. AISyLab repository, 2020. https://github.com/AISyLab/AES_HD_2.

[BPS+20] Ryad Benadjila, Emmanuel Prouff, Rémi Strullu, Eleonora Cagli, and Cécile Dumas. Deep learning for side-channel analysis and introduction to ascad database. *Journal of Cryptographic Engineering*, 10(2):163–188, 2020.

[CBT20] Lukasz Chmielewski, Ileana Buhan, and Karim Tobich. Reassure (h2020 731591) masked aes dataset, March 2020.

[CCD00] Christophe Clavier, Jean-Sébastien Coron, and Nora Dabbous. Differential power analysis in the presence of hardware countermeasures. In *Cryptographic Hardware and Embedded Systems—CHES 2000: Second International Workshop Worcester, MA, USA, August 17–18, 2000 Proceedings 2*, pages 252–263. Springer, 2000.

[CDP17] Eleonora Cagli, Cécile Dumas, and Emmanuel Prouff. Convolutional neural networks with data augmentation against jitter-based countermeasures: Profiling attacks without pre-processing. In *Cryptographic Hardware and Embedded Systems–CHES 2017: 19th International Conference, Taipei, Taiwan, September 25-28, 2017, Proceedings*, pages 45–68. Springer, 2017.

[CFG+20] Luca Cremona, William Fornaciari, Andrea Galimberti, Andrea Romanoni, and Davide Zoni. Vgm-bench: Fpu benchmark suite for computer vision, computer graphics and machine learning applications. In Alex Orailoglu, Matthias Jung, and Marc Reichenbach, editors, *Embedded Computer Systems: Architectures, Modeling, and Simulation*, pages 323–335, Cham, 2020. Springer International Publishing.

[CGL+24] Giuseppe Chiari, Davide Galli, Francesco Lattari, Matteo Matteucci, and Davide Zoni. A deep-learning technique to locate cryptographic operations in side-channel traces. In *2024 Design, Automation and Test in Europe Conference and Exhibition (DATE)*, pages 1–6, 2024.

[CK09] Jean-Sébastien Coron and Ilya Kizhvatov. An efficient method for random delay generation in embedded software. In *International Workshop on Cryptographic Hardware and Embedded Systems*, pages 156–170. Springer, 2009.

[CM24] Gaëtan Cassiers and Charles Momin. The smaesh dataset. *Cryptology ePrint Archive*, 2024.

[CRR03] Suresh Chari, Josyula R Rao, and Pankaj Rohatgi. Template attacks. In *Cryptographic hardware and embedded systems-CHES 2002: 4th International Workshop Redwood Shores, CA, USA, August 13–15, 2002 Revised Papers 4*, pages 13–28. Springer, 2003.

[DHL+21] Ba-Anh Dao, Trong-Thuc Hoang, Anh-Tien Le, Akira Tsukamoto, Kuniyasu Suzaki, and Cong-Kha Pham. Correlation power analysis attack resisted cryptographic risc-v soc with random dynamic frequency scaling countermeasure. *IEEE Access*, 9:151993–152014, 2021.

[Fre24] FreeRTOS.org. Freertos. https://github.com/FreeRTOS/FreeRTOS, 2024. Available under MIT License.

[GBTP08] Benedikt Gierlichs, Lejla Batina, Pim Tuyls, and Bart Preneel. Mutual information analysis: A generic side-channel distinguisher. In *International Workshop on Cryptographic Hardware and Embedded Systems*, pages 426–442. Springer, 2008.

[GCZ24] Davide Galli, Giuseppe Chiari, and Davide Zoni. Hound: Locating cryptographic primitives in desynchronized side-channel traces using deep-learning. In *2024 IEEE 42nd International Conference on Computer Design (ICCD)*, pages 114–121, 2024.

[GGF+24] Davide Galli, Adriano Guarisco, William Fornaciari, Matteo Matteucci, and Davide Zoni. The impact of run-time variability on side-channel attacks targeting fpgas. In *2024 31st IEEE International Conference on Electronics, Circuits and Systems (ICECS)*, pages 1–4, 2024.

[GGFZ22] Davide Galli, Andrea Galimberti, William Fornaciari, and Davide Zoni. On the effectiveness of true random number generators implemented on fpgas. In Alex Orailoglu, Marc Reichenbach, and Matthias Jung, editors, *Embedded Computer Systems: Architectures, Modeling, and Simulation*, pages 315–326, Cham, 2022. Springer International Publishing.

[GLMZ25] Davide Galli, Francesco Lattari, Matteo Matteucci, and Davide Zoni. A deep learning-assisted template attack against dynamic frequency scaling countermeasures. *IEEE Transactions on Computers*, 74(1):293–306, 2025.

[GMK16] Hannes Gross, Stefan Mangard, and Thomas Korak. Domain-oriented masking: Compact masked hardware implementations with arbitrary protection order. Cryptology ePrint Archive, Paper 2016/486, 2016.

[HMH+12] Johann Heyszl, Stefan Mangard, Benedikt Heinz, Frederic Stumpf, and Georg Sigl. Localized electromagnetic analysis of cryptographic implementations. In *Topics in Cryptology–CT-RSA 2012: The Cryptographers' Track at the*

RSA Conference 2012, San Francisco, CA, USA, February 27–March 2, 2012. Proceedings, pages 231–244. Springer, 2012.

[Inc18] NewAE Technology Inc. Cw305 artix fpga target, 2018. https://rtfm.newae.com/Targets/CW305%20Artix%20FPGA/.

[Inr] Inrevium Inc. Sasebo-gii-32.

[Kiz20] Ilya Kizhvatov. AES RD dataset, 2020. https://github.com/ikizhvatov/randomdelays-traces.

[KJJ99] Paul Kocher, Joshua Jaffe, and Benjamin Jun. Differential power analysis. In *Advances in Cryptology—CRYPTO'99: 19th Annual International Cryptology Conference Santa Barbara, California, USA, August 15–19, 1999 Proceedings 19*, pages 388–397. Springer, 1999.

[KSM21] David Knichel, Pascal Sasdrich, and Amir Moradi. Generic hardware private circuits: Towards automated generation of composable secure gadgets. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2022(1):323–344, Nov. 2021.

[MPP16] Houssem Maghrebi, Thibault Portigliatti, and Emmanuel Prouff. Breaking cryptographic implementations using deep learning techniques. In *International Conference on Security, Privacy, and Applied Cryptography Engineering*, pages 3–26. Springer, 2016.

[New20] NewAE. Chipwhisperer pro. https://wiki.newae.com/Tutorial_P1_Using_a_Custom_Trigger, 2020.

[PHJ+18] Stjepan Picek, Annelie Heuser, Alan Jovic, Shivam Bhasin, and Francesco Regazzoni. The curse of class imbalance and conflicting metrics with machine learning for side-channel evaluations. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2019(1):209–237, 2018.

[PPM+23] Stjepan Picek, Guilherme Perin, Luca Mariot, Lichao Wu, and Lejla Batina. Sok: Deep learning-based physical side-channel analysis. *ACM Computing Surveys*, 55(11):1–35, 2023.

[PWP22] Guilherme Perin, Lichao Wu, and Stjepan Picek. Exploring feature selection scenarios for deep learning-based side-channel analysis. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2022(4):828–861, 2022.

[rg14] TELECOM ParisTech SEN research group. Dpa contest (4th edition), 2013-1014. https://dpacontest.telecom-paris.fr/v4/index.php/.

[Ris20] Riscure. icwaves. https://www.riscure.com/security-tools/inspector-hardware, 2020.

[RWPP21] Jorai Rijsdijk, Lichao Wu, Guilherme Perin, and Stjepan Picek. Reinforcement learning for hyperparameter tuning in deep learning-based side-channel analysis. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2021(3):677–707, Jul. 2021. Available under MIT License.

[SZ19] Giovanni Scotti and Davide Zoni. A fresh view on the microarchitectural design of fpga-based risc cpus in the iot era. *Journal of Low Power Electronics and Applications*, 9(1), 2019.

[TBW+21] Jens Trautmann, Arthur Beckers, Lennert Wouters, Stefan Wildermann, Ingrid Verbauwhede, and Jürgen Teich. Semi-automatic locating of cryptographic operations in side-channel traces. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2022(1):345–366, Nov. 2021.

[Tcl23] Tls/ssl and crypto library. Openssl. https://github.com/openssl/openssl, 2023. Available under Apache License 2.0.

[Tec21] Pico Technology. Picoscope® 5000d series data sheet, 2021. https://www.picotech.com/download/datasheets/picoscope-5000d-series-data-sheet.pdf.

[VTM21] Aurélien Vasselle, Hugues Thiebeauld, and Philippe Maurine. Spatial dependency analysis to extract information from side-channel mixtures. In *Proceedings of the 5th Workshop on Attacks and Solutions in Hardware Security*, pages 73–84, 2021.

[WP20] Lichao Wu and Stjepan Picek. Remove some noise: On pre-processing of side-channel measurements with autoencoders. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 389–415, 2020.

[Xil20] AMD Xilinx. Ds180: 7 series fpgas data sheet: Overview, 2020. https://docs.amd.com/v/u/en-US/ds180_7Series_Overview.

[ZBHV20] Gabriel Zaid, Lilian Bossuet, Amaury Habrard, and Alexandre Venelli. Methodology for efficient cnn architectures in profiling attacks. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 1–36, 2020.

[ZGG25] Davide Zoni, Andrea Galimberti, and Davide Galli. An fpga-based open-source hardware-software framework for side-channel security research. *IEEE Transactions on Computers*, pages 1–13, 2025.